PROJECT-BASED LEARNING

PHASE-II EVALUATION

NodeSync : Fault-Tolerant Distributed Consensus Simulator For E-Commerce

Team ID: DSCPP-III-2026-T295

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T295
Semester: 3rd
Section: DS1, ML1, ML2, ML4
Domain: *AI-DS, AI-ML*

Team Members

1. *Aakshi – 2510380120*
2. *Astik Tripathi– 2510370444*
3. *Sidharth – 2512510032*
4. *Sanjam Mittal – 2510370433*

Mentor

Mr. Piyush Agarwal

Interactions completed: 8

Brief Discussion: Introduce the team, project domain, mentor, and responsibilities of the team members.

Phase-I Recap & Feedback Incorporated

Phase-I Status

In Phase-I, NodeSync was proposed as a fault-tolerant consensus simulator. Its architecture, Raft workflow, node communication, failure handling, and project structure were planned.

Feedback Received

- *Understand log entry creation and replication.*
- *Handle simultaneous client requests properly.*
- *Ensure consistency during node failures and recovery.*

Changes Incorporated

- *Clarified the log entry creation and replication process.*
- *Added handling for simultaneous client requests.*
- *Added node failure and recovery handling.*

Brief Discussion: Feedback focused on log entries, simultaneous requests, and node failures. These were incorporated through improved log handling, request synchronization, and failure-recovery mechanisms.

Problem Statement & Finalized Objectives

Problem Statement

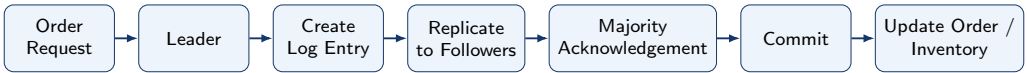
To develop a fault-tolerant distributed consensus simulator that handles concurrent client requests and maintains consistent data despite node failures and recovery.

Finalized Objectives

1. *Simulate Raft-based consensus among distributed nodes.*
2. *Handle multiple client requests concurrently.*
3. *Ensure majority-based log replication and commit.*
4. *Simulate node failures, recovery, and measure performance.*

Brief Discussion: The project focuses on fault-tolerant consensus using Raft, with scope covering concurrent client requests, log replication, node failures, and recovery. The objectives will be measured through successful consensus, consistency, and recovery.

Updated System Architecture / Workflow



Major Components

- *Distributed Node Network Simulation*
- *Raft Consensus Log Replication*
- *Failure Detection Recovery*

Changes Since Phase-I

- *Added structured client–leader–follower communication.*
- *Concurrent client request handling.*
- *Refined log replication, commitment, and failure recovery.*

Brief Discussion: The updated architecture connects the Client, Network, Raft Nodes, and Storage modules. Client requests pass through the Network to the Leader, which replicates log entries to Followers and commits them through Storage. The Network also handles communication, delays, and failures between nodes.

Module Design & Functional Requirements

Major Modules

1. *Client Management – Processes client requests.*
2. *Raft Consensus – Handles leader election and replication.*
3. *Network Simulation – Manages node communication and failures.*
4. *Data Storage – Maintains logs and committed data.*

Functional Requirements

- *Receive and queue client requests.*
- *Conduct leader election using Raft.*
- *Replicate and commit logs consistently.*
- *Simulate node failures and recovery.*

Module Dependency

Client → Load Balancer → Raft → Storage.

Brief Discussion: Explain what each module does and how it contributes to the complete system.

Technology Stack & Development Methodology

Technology Stack

- Programming: *C++ 17/C*
- Database: *In-memory storage - No external DBMS*
- Framework: *STL/CMake*
- Development Tool: *Visual Studio Code*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis – Finalize implementation requirements.
2. System Design – Finalize architecture and module interaction.
3. Module Development – Implement core NodeSync modules.
4. Integration – Integrate Raft, network, and storage modules
5. Testing & Validation– Test consensus, failures, and performance.

Brief Discussion: C++17 and STL support efficient implementation, while CMake and VS Code simplify development. The team follows a structured design, development, integration, and testing process.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	*Graphs, Hash Maps, Queues, Priority Queues, Arrays, Searching & Sorting.*	*Network & Message Simulation, Node State Management.*
OOPs with C++	*Classes & Objects, Encapsulation, Abstraction, Inheritance, Polymorphism, STL.*	*Node, Network, Raft Consensus & E-Commerce Module.*

Important: Only the concepts that have been implemented and used in the current project modules are listed; planned or unused concepts are excluded.

Brief Discussion: Graphs and queues are used for network communication and message handling, while arrays and linked lists support log and node management. C++ classes, encapsulation, abstraction, and STL are used to implement nodes, network operations, and Raft consensus.

Development & Implementation Progress

Module	Status	Work Completed
Node & Network Simulation	Completed	*Node communication implemented*
Raft Consensus	Completed	*Leader election and log replication developed*
Client Request Handling	In Progress	*Concurrent requests being implemented*
Failure & Performance	Pending	*Failure simulation and metrics planned*

Overall Project Completion: *60%*

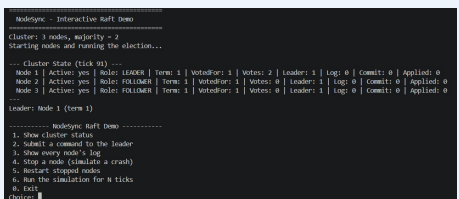
Brief Discussion: Node creation and basic network communication are completed. Raft consensus and log replication are in progress. Concurrent client requests, failure simulation, and performance metrics are pending.

Working Prototype / Demonstration

Implemented Features

- *1. Node Network Simulation*
- *2. Raft Leader Election*
- *3. Log Replication Commitment*
- *4. Storage Operations*

Prototype / Output



Brief Discussion: The NodeSync prototype demonstrates distributed node communication, Raft leader election, log replication, majority-based commitment, and log storage operations. Automated tests validate the behavior of these components.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Storage Operations	*Entries append, retrieve and truncate correctly*	*Verify using test output*	*Verified*
Append Entries Protocal	*Follower validates log index and term*	*Dedicated replication tests available in project history*	*Verified*
Log Conflict Resolution	*Conflicting uncommitted entries are handled correctly.*	*Fill after running the current test.*	*Verified*

Results / Validation

Unit tests validate storage and log operations. Replication tests cover AppendEntries, duplicate or stale messages, and follower catch-up. Commitment tests check replication and quorum-related behavior. Final pass/fail status should be recorded after running the test suite on the current working tree.

Brief Discussion: Tests evaluate storage operations, log consistency, replication, and majority-based commitment. They help identify incorrect log updates and verify expected consensus behavior.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: *AAKSHI1206/nodesync-consensus-simulator*
- README.md: *Updated*
- Contributors: *All Members Added*
- Repository Status: *Updated*
- Branches: *NA*

Development Activity

- *Regular commits maintained*
- *Meaningful commit messages*
- *Individual contribution visible*
- *Latest implementation uploaded*
- *Documentation maintained*

Brief Discussion: Show repository activity, README, contributor list and commit history during the presentation.

Individual Contribution

Member	Role	Technical Contribution	Commits
Aakshi	*Raft & Message Handling*	*Implemented the Raft election engine, voting logic, and message model; contributed to queue implementation and election tests.*	*11*
Astik Tripathi	*Network & Load Balancer*	*Implemented the C-based log structure, network communication, and the basic Load Balancer and Server modules.*	*11*
Sidharth	*NetworkIntegration & Simulation*	*Implemented the C linked-list structure, multi-node communication integration, and tick-based cluster simulation.*	*11*
Sanjam Mittal	*Storage & DSA Validation*	*Developed the Storage abstraction, Node implementation, storage tests, and README documentation.*	*11*

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Understanding Raft consensus and leader election.*
- *Coordinating communication between multiple nodes.*
- *Managing log entries and concurrent requests.*

Solutions / Learning

- *Adopted modular design for easier integration and debugging.*
- *Learned Raft consensus, leader election, log replication, and node communication.*

Phase-III Roadmap

1. *Complete client handling and failure modules.*
2. *Integrate all NodeSync components.*
3. *Test consensus, failures, and performance.*
4. *Optimize and validate the system.*
5. *Finalize documentation and project report.*

Expected Outcomes & References

Expected Final Outcomes

- *Complete working fault-tolerant consensus simulator.*
- *Measurable consensus, recovery, and performance results.*
- *Practical application in distributed e-commerce systems.*
- *Scope for scalability and real-world deployment.*
- *Potential for further research and improvement.*

References

1. **Research Paper:** *Ongaro, D. & Ousterhout, J. – In Search of an Understandable Consensus Algorithm, USENIX ATC 2014.*
2. **Raft Standard / Algorithm:** *Raft Consensus Algorithm – Official Raft website.*
3. **Official Documentation:** *C++ Reference / Standard Library – cppreference.*

Brief Discussion: The project will deliver a working Raft-based consensus simulator with measurable performance results, supported by Raft and distributed-systems resources.

Thank You

Questions & Suggestions